

A Stateful Executable Proof: Governing an HTTP Echo Service Lifecycle Across Eight Languages

Werner Kasselmann
Verivus OSS
werner@verivus.com

May 31, 2026

Abstract

A companion proof in this repository showed that a DAG-TOML contract can govern an instant, deterministic, stdout-only program across languages. That contract could be checked by running each program to completion and diffing stdout. This paper reports the follow-up: the same DAG-TOML machinery—contract declaration, implementation DAG, traceability, readiness gate, evidence matrix, and executable witnesses—pointed at a contract that *cannot* be checked that way. The contract is stated over eight languages, of which seven (Go, Node/JavaScript, TypeScript, Python, C, Rust, Java) must each bind a port, stay alive, echo a `POST` body byte-for-byte, answer repeated requests, and shut down gracefully on `SIGTERM`—finishing any in-flight request, releasing the port, and exiting 0. The eighth language, AWK, is the declared signal boundary: gawk exposes no clean script-level `SIGTERM` handler, so it is held to the lifecycle contract only as a `SKIP-with-rationale` and its echo is left unasserted. On the runner used for this paper the load-bearing witness reported seven `PASS` outcomes, one `SKIP` for AWK, and zero `FAIL`. A negative-control witness confirms the graceful-shutdown check is non-vacuous: it catches both a server that ignores `SIGTERM` and a server that drops an in-flight response while exiting 0. The contribution is not a benchmark or a production framework. It is a compact, falsifiable artifact that names precisely which parts of the proof structure had to grow for a process *lifecycle*—a lifecycle-aware witness harness, the `MEASURED` result word used for real, and a port-as-shared-resource discipline—and which dynamic guarantees the static validators still cannot see. A pre-build runtime spike, reported in full, also corrects one design assumption: Java’s bundled `HttpServer` cannot set `SO_REUSEADDR`, which fixes the run order rather than the claim.

1 Introduction

Artifact-centered papers are easiest to review when claims, commands, and files line up directly, and artifact-evaluation guidance makes the same point operationally: reviewers need enough structure to connect a paper’s claims to executable scripts, source files, expected outputs, and limitations [4, 8]. A companion paper applied that structure to the smallest nontrivial instance: a “Hello, world!” contract requiring exact stdout bytes and exit 0. That proof is honest but limited in one specific way—its contract is satisfiable by a program that runs to completion, so the witness is a single stdout `cmp`.

Real assurance targets are not like that. Services bind ports, stay resident, answer over the network, and must shut down without dropping work. This paper asks whether the same inspectable,

falsifiable DAG-TOML proof structure survives that jump. The contract under study, declared in `non-trivial-proof/proof-bundle`, is an HTTP echo service. Each PASS-candidate implementation must bind `127.0.0.1:8080`, return `200 OK` whose body is the exact bytes of the request body, answer at least two sequential requests on one process, and, on `SIGTERM`, finish any in-flight request, release the port, and exit 0. AWK is exempted from this lifecycle contract as the declared C06 boundary (Section 6 and the C06 row of Table 1).

The jump forces machinery the `stdout` proof never needed. The verify unit must background a server, poll for readiness, drive it over the network, signal it, and capture the post-signal exit status with guaranteed teardown. A bound port is shared mutable state on the runner, so the language runs must be serialized and the port proven released between them. Timing becomes a first-class observation, which is where the fourth result word, `MEASURED`, is used for the first time in this repository. And graceful shutdown is only meaningful against a negative control, so the proof includes a witness whose PASS means the gate correctly classified a non-graceful server as FAIL.

This remains a minimal proof, not a performance study. There is no throughput, latency percentile, or concurrency-under-load claim. The goal is the same as before: an artifact a reviewer can inspect in one sitting, with every dynamic guarantee tied to an executable witness and every non-claim stated.

2 Artifact

The proof pack lives under `non-trivial-proof/proof-bundle`. Its normative files are:

- `contract_declaration.toml`: contracts C01 through C06.
- `implementation_dag.toml`: ten units, all tier 1.
- `traceability.toml`: three chains (service lifecycle, graceful-vs-kill, AWK boundary).
- `review_readiness.toml`: one gate for the service-witness pack.
- `evidence_matrix.toml`: five claims mapped to seven evidence artifacts.
- `run_service_contract.sh`, `detect_graceful_shutdown.sh`, and `detect_awk_boundary.sh`: executable witnesses.

The eight servers are `src/go/server.go`, `src/node/server.js`, `src/typescript/server.ts`, `src/python/server.py`, `src/c/server.c`, `src/rust/server.rs`, `src/java/Server.java`, and `src/awk/server.awk`. Two non-graceful negative controls live in `src/controls/`. The echo payload is a fixed 56-byte JSON file, `payload.json`.

Eight languages, six runtimes, one boundary. The firm headline number is eight languages. “Six runtimes” is a derived figure under an explicit definition: a runtime is a distinct language execution stack. TypeScript folds into the V8/Node stack it shares with JavaScript, and AWK is the C06 boundary rather than a passing runtime, leaving six distinct stacks—Go, V8/Node (JS and TS), CPython, the JVM, natively-compiled C, and natively-compiled Rust. We flag the soft edge honestly: C and Rust are both bare native binaries over `libc`, so a stricter “managed-runtime” definition would merge them and reach five. The paper therefore leads with the number that is directly verifiable from the source tree.

3 Contract and Witnesses

Table 1 lists the six declared contracts and the local witness that supports each. C01 is the load-bearing lifecycle contract. C02 through C05 narrow separable obligations that depend on C01. C06 is the declared AWK signal boundary with an explicit non-claim.

Contract	Domain	Witness and scope
C01	service_lifecycle	<code>run_service_contract.sh</code> ; bind, byte-exact echo, exit 0 on SIGTERM, port released. Load-bearing.
C02	readiness	Connectable on 127.0.0.1:8080 within 5000 ms; <i>time-to-ready</i> reported as MEASURED.
C03	echo_fidelity	Status exactly 200, body <code>cmp</code> -equal to the request body, <code>Content-Length</code> equal to body length.
C04	signal_handling	In-flight request completes, exit 0 within 3000 ms, port re-bindable; <i>time-to-shutdown</i> MEASURED. Backed by the negative control.
C05	statefulness	One long-lived process answers at least two sequential requests.
C06	signal_boundary	<code>detect_awk_boundary.sh</code> ; gawk has no clean script-level SIGTERM handler, so AWK C04 is a declared SKIP; its echo is UNASSESSABLE.

Table 1: Contract-to-witness inventory.

The C04 in-flight check is made race-free by an explicit synchronization point rather than a timing guess. The test-only `?delay_ms=` request path makes a handler flush its status line and `Content-Length` headers immediately, then sleep before writing the body. The witness blocks on reading those headers off the socket (proof the handler has been entered and committed its response), and only then sends SIGTERM. “The request is in flight” is thus an observed fact, not an assumption about how fast the signal arrives. The injected delay is fixed at 1000 ms, strictly less than the 3000 ms shutdown deadline, so a correct graceful server can both finish the request and exit in time; the only way to fail is to actually drop the in-flight response.

4 Implementation DAG

The implementation DAG is a fan-in graph, but here the fan-in is load-bearing for correctness rather than tidiness: a single verifier must own the shared port. Seven layer-0 build/prepare units fan into one layer-1 verification unit that runs each server one at a time on :8080. Two independent layer-0 leaves run the behavioural sidecar witnesses.

```

U01 go-server      \
U02 node/ts-server \  (one Node/V8 unit, two source files)
U03 python-server  \
U04 c-server       > U08 verify-service-contract (Java first)
U05 rust-server    /
U06 java-server    /
U07 awk-server     /

U09 verify-graceful-vs-kill  (independent leaf)
U10 verify-awk-boundary     (independent leaf)

```

The computed values checked by the repository’s implementation-DAG validator are:

- ten units;

- layer counts {0: 9, 1: 1};
- entry points U01..U07, U09, U10 (empty `depends_on`);
- leaf nodes U08, U09, U10 (empty `blocks`); U09 and U10 are simultaneously entry points and leaves;
- critical path U05 -> U08 with node-weighted critical-path LOC 327.

Run order is not arbitrary. U08 runs Java first, on a pristine port. The reason is a measured runtime fact, not a preference; it is discussed in Section 9.

5 The Witness Harness (the part that grew)

The stdout proof’s harness ran each program to exit and compared bytes. This harness adds five primitives the stdout proof never needed: PID capture into a dedicated process group (so a failed assertion never orphans a daemon or its children on the port), readiness polling with a deadline, signal delivery, exit-after-signal capture, and guaranteed teardown on every exit path. Port 8080 is treated as a global, mutable, singleton resource: the verifier serializes the language runs, fails fast to SKIP if the port is already occupied, and proves the port is re-bindable after each server exits using an *independent* re-bind probe that itself sets `SO_REUSEADDR` and retries for up to two seconds to absorb `TIME_WAIT`. Testing only that the *server* set the option is not enough, because the probe is a fresh socket that can hit `TIME_WAIT` on its own.

6 Observed Execution

All commands were run from the repository root on May 31, 2026. The full toolchain was present: go 1.26.3, node v24.15.0, python3 3.13.13, cc (gcc) 15.2.0, rustc 1.90.0, java 25.0.3 (runtime only; no javac), and gawk 5.3.2, with port 8080 free.

Command	Exit	Observed result
<code>bash.../run_service_contract.sh</code>	0	7 pass, 1 skip, 0 fail. Go, Node, TS, Python, C, Rust, Java passed; AWK skipped as the declared C06 boundary.
<code>bash.../detect_graceful_shutdown.sh</code>	0	Both non-graceful controls caught as C04 FAIL (ignore-SIGTERM and drop-in-flight).
<code>bash.../detect_awk_boundary.sh</code>	0	gawk listener up; echo UNASSESSABLE; SIGTERM yields exit 143, confirming no clean handler.
<code>validate_implementation_dag.py...</code>	0	Passed; 10 units, layers {0:9,1:1}, critical-path LOC 327.
<code>--check-paths-exist</code>	0	Passed; 29 entities, path checks enabled.
<code>validate_traceability.py...</code>	0	
<code>--check-paths-exist</code>	0	
<code>validate_review_readiness.py... (three files)</code>	0	Contract-declaration, readiness-gate, and evidence-matrix passed.

Table 2: Observed command outcomes.

Table 3 gives the per-language lifecycle results from one representative run. The two timing columns are MEASURED numbers, not gates (Section 7).

The roughly 1000 ms shutdown for six runtimes is the injected 1000 ms in-flight delay being honoured:

Language	Result	time-to-ready	time-to-shutdown
Java	PASS	566 ms	24 ms †
Go	PASS	4 ms	1070 ms
Node	PASS	48 ms	1024 ms
TS	PASS	71 ms	1024 ms
Python	PASS	47 ms	1026 ms
C	PASS	3 ms	1002 ms
Rust	PASS	3 ms	1001 ms
AWK	SKIP	—	C06 boundary; echo UNASSESSABLE

Table 3: Per-language lifecycle (one representative run; MEASURED timings vary run to run).

the server finishes the delayed request before exiting. † Java is the exception at 24 ms because `server.stop(2)` interrupts the handler’s sleep, so Java writes the full body immediately instead of waiting out the artificial delay. The in-flight response still arrives complete and byte-exact and the exit code is 0, so C04 holds—this is honest graceful completion, in direct contrast to the drop-in-flight control below.

7 The measured Result Word

The companion stdout proof used four result words but reported no MEASURED values, because nothing in that contract was a timed observation. This proof is the first in the repository to use the fourth word for real, and it is disciplined about what the word means. A MEASURED number—*time-to-ready*, *time-to-shutdown*, or bytes echoed—is descriptive evidence, never a PASS on its own. “Within the deadline” is a *separate* PASS/FAIL contract: C02 fails if the server is not connectable within 5000 ms, and C04 fails if it has not exited within 3000 ms, regardless of the exact measured figure. The timings in Table 3 vary from run to run; that variance is precisely why they are reported as measurements and not promoted to gates. (A scope note: the claim of “first MEASURED use” is currently scoped to this papers repository, not the wider organization.)

8 Negative Control: Graceful versus Killed

A graceful-shutdown check that only inspects the exit code is vacuous: a server can `exit(0)` the instant it receives `SIGTERM` and look clean while dropping every in-flight request. The C04 definition is therefore the stronger one—an in-flight request must *complete*—and `detect_graceful_shutdown.sh` proves the gate enforces it on both axes. It drives two deliberately broken Go servers:

- **Control A** installs no `SIGTERM` handler. The witness must find it still running after the deadline and escalate to `SIGKILL`; that is a C04 FAIL.
- **Control B** calls `exit(0)` on `SIGTERM` while a slow request is mid-flight. Its exit code is a clean 0, but the client receives a truncated body. The witness must read the truncated response and classify it FAIL anyway.

On this runner both controls were caught: Control A timed out and needed `SIGKILL`; Control B exited 0 but delivered 0 of 56 expected body bytes. PASS for this witness means the C04 check discriminates graceful from non-graceful on the exit-code axis *and* the in-flight-completion axis. This is the single most important honesty witness in the bundle, and it is the lifecycle analogue of

the stdout proof’s “a divergent implementation would fail” control.

9 A Measured Runtime Correction: Java and SO_REUSEADDR

Service proofs interact with the operating system in ways a static design document can get wrong, so the design was checked against a pre-build runtime spike before any server was written. The spike overturned one design assumption and is reported here in full because correcting it is part of the result.

The draft design assumed Java’s bundled `com.sun.net.httpserver.HttpServer` would carry the same `SO_REUSEADDR` guarantee as the other runtimes. It does not. On this runner (OpenJDK 25.0.3, Linux, `tcp_fin_timeout=60`) the spike established that `HttpServer` sets no `SO_REUSEADDR` and exposes no API to set it: neither `HttpServer.create(addr, backlog)` nor the unbound `create()` followed by `bind()` surfaces the underlying `ServerSocket`, so `setReuseAddress(true)` before `bind` is unreachable. Once the server has served a connection and closed it server-side, the port holds a `TIME_WAIT` entry and a fresh `HttpServer` cannot re-bind it for the full timeout window; every spike run ended in `BindException: Address already in use`. A plain `ServerSocket` with `setReuseAddress(true)` binds the same port immediately.

The correction is a run-order constraint, not a weakened claim. Java is scheduled *first* in the serialized verify order, on a pristine port; its pre-flight is a `TIME_WAIT`-aware probe rather than the connectivity probe used for the other six. A lingering foreign `TIME_WAIT` when Java’s turn comes is recorded as a legitimate `SKIP`, never a false `FAIL`. Java’s own port-release check still passes, because that assertion is made by the independent `SO_REUSEADDR`-setting probe socket, not by Java re-binding. The other six runtimes set `SO_REUSEADDR` (Go and Node by default, C via `setsockopt`, Rust via a small `extern "C"` FFI block, Python via `allow_reuse_address`) and tolerate a prior `TIME_WAIT`.

10 PASS, SKIP, FAIL, and MEASURED

The proof uses the four result words narrowly.

- `PASS` means the witness exited 0 and reported the checked lifecycle condition held: ready, byte-exact echo over at least two requests, in-flight completion, exit 0 within the deadline, and port released.
- `SKIP` means a declared check was not performed because a toolchain was absent, the port was occupied, or the runtime is the declared C06 boundary (AWK). `SKIP` is not evidence of conformance.
- `FAIL` means a bind failure, a missed deadline, a non-200 status, a body mismatch, a dropped in-flight response, a nonzero exit on `SIGTERM`, or a port not released.
- `MEASURED` is reserved for the timing and byte observations of Section 7; it is never promoted to a `PASS`.

11 Claim Audit

Table 4 maps the paper’s claims to evidence, separating direct observation from inference.

Claim	Evidence source	Status	Counterexample boundary
The proof pack is structurally complete for this example.	the five <code>*.toml</code> files and the repository validators.	Directly observed.	Does not imply production readiness.
The seven available servers satisfy the C01–C05 lifecycle on this runner.	<code>run_service_contract.sh</code> and the observed run.	Directly observed; AWK skipped as the C06 boundary.	A runner missing a toolchain or port would produce more SKIPS.
The C04 graceful check is non-vacuous.	<code>detect_graceful_shutdown.sh</code> and the two controls.	Directly observed.	Does not prove zero drops under arbitrary concurrency.
Java’s <code>HttpServer</code> cannot set <code>SO_REUSEADDR</code> ; Java is scheduled first.	the pre-build spike and the witness pre-flight.	Directly observed on this JDK and runner.	Does not characterize other JDKs or HTTP libraries.
The AWK signal boundary is real and its echo is unasserted.	<code>detect_awk_boundary.sh</code> ; <code>SIGTERM</code> exit 143.	Directly observed plus a declared non-claim.	Does not prove <code>gawk /inet</code> cannot echo byte-exact.
Artifact organization follows current review expectations.	ACM badging, POPL artifact guidance, and SIGSOFT standards [4, 8, 1].	Cited.	No formal artifact badge is claimed.

Table 4: Claim-to-evidence audit.

12 Related Work

The proof’s framing is an executable specification used as a consistency check rather than a benchmark, a line of work with precedent in trace specifications, where formal specifications are turned into executable models [6]. The lifecycle contract itself encodes widely-recommended operational practice: disposable processes that start fast and shut down gracefully on `SIGTERM`, finishing in-flight work before exit, is the disposability principle of the twelve-factor methodology [11], and treating non-graceful termination as a first-class failure mode to be tested echoes stability-pattern guidance for production services [7].

The artifact framing follows research-artifact guidance rather than benchmark methodology. ACM distinguishes artifact evaluation, availability, and result validation [4]; POPL’s artifact guidance asks authors to map scripts and source files to paper claims [8]; and SIGSOFT’s empirical standards emphasize specific, technical review checklists for software-engineering research [1, 9].

13 Threats to Validity

The validity structure separates what was observed from the broader claims one might draw [10, 5].

Construct validity. The proof measures whether a stateful lifecycle contract is represented and checked, not whether DAG-TOML is sufficient for a large assurance pipeline. The echo service is intentionally minimal: it is not a benchmark, not RFC-conformant HTTP, and has no TLS or keep-alive guarantees beyond the test.

Internal validity. The strongest internal risks were addressed before the reported run. The in-flight check uses an explicit header-flush synchronization point rather than a timing guess; the port-release check uses an independent `SO_REUSEADDR` re-bind probe rather than a mere listener-absence check; and the byte-exact comparison checks `Content-Length` as well as the body. Each of these was a real defect caught and corrected during review.

External validity. The proof covers eight small servers on one POSIX runner. `SIGTERM/SIGKILL` semantics assume POSIX; Windows is out of scope. Port 8080 is a runner-global assumption, and a hostile environment racing for the port yields a legitimate SKIP.

Conclusion validity. The results are categorical witness outcomes and descriptive MEASURED timings, not statistical estimates. The correct conclusion is that the declared witnesses passed, skipped, or failed as reported on this runner.

14 What the Static Validators Do Not Check

The five DAG-TOML validators check structure, not runtime. They confirm that C01 is declared, wired to a test, and that the witness scripts exist; they cannot confirm that a server ever bound, echoed, or shut down. Every dynamic guarantee lives only in the shell witnesses and is surfaced into Section 6. The readiness-gate validator, in particular, requires a gate identifier, an artifact class, and one of a small set of content fields, and resolves the artifact-class link and the required-document paths; it does *not* interpret the human-readable pass and block conditions, which are enforced only by running the witness. Path existence is itself opt-in: every validator invocation in this paper passes `-check-paths-exist` with a repository root, or the existence guarantee would be hollow. There is no validator hook for MEASURED; it is a paper-level reporting discipline. This gap is the natural bridge to later follow-ups in the same arc, especially handing verification to enterprise static-analysis tools, and is noted here as future work without overclaiming.

15 arXiv and Artifact Packaging Notes

The public artifact repository for this paper is <https://github.com/verivus-oss/agent-assurance-papers>; the specification and validators are at <https://github.com/verivus-oss/agent-assurance>. The package is intentionally plain: `article`, `pdflatex`, `BibTeX`, `natbib`, and standard packages. arXiv processes TeX submissions automatically, expects authors to inspect the generated PDF, and asks that extraneous files such as logs and aux files be removed from the source package [2, 3]. An arXiv source package for this paper should include only `main.tex`, `references.bib`, and a precomputed `main.bbl`; the bundle code can be linked or attached as ancillary material.

16 Limitations and Non-Claims

This paper does not claim throughput, latency, or concurrency-under-load results; RFC-conformant HTTP, TLS, or keep-alive guarantees; correctness on non-POSIX runners; that gawk `/inet` cannot

achieve byte-exact echo (only that this bundle does not assert it); or production readiness. The exact claim is narrower: the repository contains a multi-language stateful-service proof whose lifecycle contract, implementation graph, traceability chains, and review gate can be checked against executable witnesses; whose graceful-shutdown check is shown non-vacuous by negative controls; whose one declared signal boundary is explicit; and whose observed outcomes are reproducible by running the listed commands in an environment with the same toolchains.

17 Conclusion

The stdout proof showed that a DAG-TOML contract can govern identical output bytes across languages. This proof shows the same structure governing an identical *lifecycle*—ready, echo, graceful shutdown, port release—across seven of eight languages (six runtimes), with the eighth, AWK, held as the one declared signal boundary rather than to the lifecycle contract. What had to grow is named precisely: a lifecycle-aware witness harness, the MEASURED result word used for real, a port-as-shared-resource discipline, and a negative control that keeps the graceful-shutdown check honest. What did not change is the proof structure itself. The artifact is not evidence about performance or production assurance; it is a compact, executable proof that a stateful service contract can be tied to contracts, DAG nodes, code paths, witnesses, and review gates, and that the parts a static validator cannot see are stated rather than hidden.

References

- [1] ACM SIGSOFT. Empirical standards for conducting and evaluating research in software engineering, 2026. URL <https://www2.sigsoft.org/EmpiricalStandards/>.
- [2] arXiv. Submit TeX/LaTeX, 2026. URL https://info.arxiv.org/help/submit_tex.html.
- [3] arXiv. TeX live at arxiv, 2026. URL <https://info.arxiv.org/help/faq/texlive.html>.
- [4] Association for Computing Machinery. Artifact review and badging version 1.1, August 2020. URL <https://www.acm.org/publications/policies/artifact-review-and-badging-current>.
- [5] Robert Feldt and Ana Magazinius. Validity threats in empirical software engineering research: An initial survey. In *Proceedings of the 22nd International Conference on Software Engineering and Knowledge Engineering*, pages 374–379, 2010.
- [6] Daniel M. Hoffman and Richard T. Snodgrass. Trace specifications: Methodology and models. *IEEE Transactions on Software Engineering*, 14(9):1243–1252, 1988. doi: 10.1109/32.6168.
- [7] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018. ISBN 9781680502398.
- [8] POPL 2023 Artifact Evaluation Committee. POPL 2023 artifact evaluation, 2023. URL <https://popl23.sigplan.org/track/POPL-2023-artifact-evaluation>.
- [9] Paul Ralph, Sebastian Baltes, Domenico Bianculli, Yvonne Dittrich, Michael Felderer, Robert Feldt, Antonio Filieri, Carlo A. Furia, Daniel Graziotin, Peng He, et al. Empirical standards for software engineering research, 2020. URL <https://arxiv.org/abs/2010.03525>.

- [10] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.
- [11] Adam Wiggins. The twelve-factor app: Disposability, 2017. URL <https://12factor.net/disposability>.